Kevin Küster

Zusammenfassung: Software Engineering

Software Engineering

Software Engineering

- Programmierung
  - ↳ im kleinen: Ein Entwickler, überschaubar, bekannte Techniken
  - ↳ im großen: Teams, Jahre, Millionen LOC, Kommunikation zentral, Anforderungen variabel → Probleme im Großen können nicht durch Lösungen im kleinen gelöst werden
- IEEE (Def.): Die Anwendung eines systematischen, disziplinierten, quantifizierbaren, Ansatzes auf die Entwicklung, den Betrieb und die Wartung von Software; d.h. die Anwendung der Ingenieurwissenschaften auf Software.

⇒ Ingenieurmäßiges Vorgehen → planbar, nachvollziehbar, reproduzierbar

Softwarequalitätsmerkmale

- Externe Qualität: für Benutzer sichtbar
- Interne Qualität: für Entwickler; Architektur, Wartbarkeit
- ISO 9126:
  - ↳ Funktionalität: Angemessenheit, Richtigkeit, Interoperabilität, Sicherheit
  - ↳ Zuverlässigkeit: Reife, Fehlertoleranz, Wiederherstellbarkeit
  - ↳ Effizienz: Zeitverhalten, Verbrauchsverhalten (Ressourcen)
  - ↳ Änderbarkeit (Wartbarkeit): Analysierbarkeit, Modifizierbarkeit, Stabilität, Testbarkeit
  - ↳ Übertragbarkeit: Anpassbarkeit, Installierbarkeit, Konformität, Austauschbarkeit
- Messen:
  - ↳ Zuverlässigkeit → Testlaufzeit + Fehlerzahl
  - ↳ Benutzbarkeit → Usability-Test
  - ↳ Effizient: Benchmarks

Prozessqualität [CMM(I)]

CMMI (Capability Maturity Model Integration) - 5 Reifegrade:

(i) Initial: Chaotisch, Erfolg hängt von Einzelperson ab

(ii) Managed: Grundlegendes Projektmanagement

(iii) Defined: Standardprozess definiert, organisationsweite Anwendung

(iv) Quantatively Managed: Prozess wird gemessen und kontrolliert

(v) Optimizing: Kontinuierliche Prozessverbesserung

Gebrauchstauglichkeit (Usability / UX)

- Usability (ISO 9241-11): Ausmaß, in dem ein Produkt von bestimmten Benutzern genutzt werden kann, um bestimmte Ziele effektiv, effizient und zufriedenstellend zu erreichen.
- User Experience (UX): Gesamtes Ergebnis vor, während und nach Nutzung
- Herausforderungen: Unterschiedliche Nutzergruppen, kontextabhängig, muss früh im Entwicklungsprozess berücksichtigt werden, Nutzerbeteiligung
- Negative Auswirkungen fehlender Gebrauchstauglichkeit:

  Produktivitätsverlust, Nutzerakzeptanz sinkt, Supportkosten, Imageverlust, Schulungsaufwand

Vorgehensmodelle

Grundtypen

- Wasserfallmodell: Klare Reihenfolge, Ergebnisdokument
  - ↳ Phasen: Anforderung → Entwurf → Implementierung → Test → Betrieb
- V-Modell: Erweitert Wasserfall um korrespondierende Testphasen

  ⇒ Standard der Bundeswehr / Verwaltung
- Spiralmodell (Boehm): Iteration mit Risikoanalyse, a priori nicht festgelegte Zyklenanzahl
- Agile Modelle: Fokus auf Kommunikation, funktionierende Software, Reagieren auf Änderungen

V-Modell / V-Modell XT

V-Modell: Phasenmodell + korrespondierende Tests

(Anforderungen ↔ Abnahmetest, Entwurf ↔ Integrationstest)

↳ pro: Umfassend, QS integriert, Nicht-SW-Komponenten erfasst

↳ contra: Starke Reglementierung, „Papierflut", geringe Flexibilität

Einsatz: Sicherheitskritische Systeme, Bw, öffentliche Verwaltung

V-Modell XT: Erweiterung um Tailoring (Anpassung an Projekt), iteratives Vorgehen, Vorgehensbausteine

Scrum

- Rollen
  - ↳ Product Owner: Pflegt Produkt Backlog, priorisiert Anforderungen
  - ↳ Scrum Master: Sorgt für korrekten Scrum-Prozess, beseitigt Hindernisse, Kein Projektleiter (!)
  - ↳ Team: Selbstorganisiert, setzt Anforderungen um
- Artefakte
  - ↳ Product Backlog: Priorisierte Liste aller Anforderungen (User Stories) mit Aufwandschätzung
  - ↳ Sprint Backlog: Aufgaben für den aktuellen Sprint
  - ↳ Burndown Chart: Visualisierter Restaufwand
  - User Story: „Als [ROLLE] will ich [FUNKTION], damit [NUTZEN]."
- Events / Meetings
  - ↳ Sprint Planning: Auswahl der Items aus Backlog; Team erstellt Sprint Backlog (Planning Poker)
  - ↳ Daily Scrum: 15 min Stand-up, 3 Fragen: gestern/heute/Hindernisse
  - ↳ Sprint Review: Ergebnisse vorstellen, Abnahme
  - ↳ Sprint Retroperspective: Prozessverbesserung (Kaizen)
- Sprint
  - ↳ Aktive Umsetzung/Arbeitsphase
  - ↳ 1-4 Wochen
  - ↳ Keine neuen Anforderungen während Sprint
- Priorisierung
  - ↳ Zwingend notwendig
  - ↳ Weil Sprints zeitbedingt → nur wichtigste Items umgesetzt
  - ↳ Kriterium: Geschäftswert / Kundennutzen

Kein Product Owner verfügbar? → Problem; PO muss Backlog pflegen + priorisieren

⇒ Ohne PO keine Anforderung was gebaut werden soll

XP (Extreme Programming)

Grundideen: Kein separates Design, Code = Dokumentation, Tests vor Code (TDD), Pair Programming, 40h-Woche, Kunde im Team, kurze Iteration, Refactoring

↳ pro: Flexibel, schnelle Ergebnisse, gute Kommunikation

↳ contra: keine Docs, Nur kleine Projekte, Kunde muss permanent verfügbar sein

Agiles Manifest

↳ Werte (Beck, Fowler et al. 2001):

(i) Individuen & Interaktionen > Prozesse & Werkzeuge

(ii) Funktionierende Software > Umfangreiche Doc

(iii) Zusammenarbeit mit Kunde > Vertragsverhandlungen

(iv) Reagieren auf Änderungen > Verfolgen eines Plans

Qualitätssicherungen & Tests

Konstruktive vs. Analytische QS

Konstruktive QS: Betrifft Prozess

↳ Vorgehensmodelle, Werkzeuge, Schulung, Arbeitsumgebung

⇒ Fehler von vornherein Vermeiden

Analytische QS: Betrifft Produkt → „Test":

▷ Analysierende Verfahren (statisch): Code/Dokumente ohne Ausführung prüfen

↳ Inspektion/Review/Walkthrough; Statische Analyse; Verifikation

↳ Erkennt keine falschen Parametertypen bei dynamischer Typisierung

▷ Testende Verfahren (dynamisch): Code mit konkreten Eingaben ausführen

↳ dynamischer Test; symbolischer Test; Schreibtischtest

White-Box vs. Black-Box Testing

▷ White-Box (Strukturtest): Interne Struktur bekannt

↳ Tests basieren auf Code

↳ Überdeckungskriterien: Anweisungen (C0), Zweige (C1), Bedingungen (C2), Mehrfachbedingungen (C3), Pfade (C4)

↳ Zweigüberdeckung (C1) = realistisches Minimalkriterium

▷ Black-Box (Funktionstest): Interne Struktur unbekannt

↳ Test basieren auf Spezifikation

↳ Äquivalenzklassen: Eingabebereich in Klassen einteilen; repräsentative Werte wählen

↳ Grenzwertanalyse: Werte an den Grenzen der Äquivalenzklassen testen

↳ größte Fehlerquelle

Kann testen alle Fehler finden? → Nein

⇒ Testen kann nur die Anwesenheit, nicht die Abwesenheit von Fehlern zeigen (Dijkstra)

⇒ Vollständige Tests i.d.R. unmöglich → unendlich viele Inputs

Testebenen

▷ Komponententest (Unit Test): Einzelne Module/Klassen isoliert testen

▷ Integrationstest: Zusammenspiel mehrerer Komponenten

▷ Systemtest: Gesamtsystem gegen Anforderungen

▷ Abnahmetest: Durch Kunden, gegen Vertrag/Anforderungen

Aufwandsschätzung

Schätzverfahren

▷ Pricing to win: Kosten = Budget des Kunden; Anforderungen anpassen

▷ Experteneinschätzung: Vergleich mit ähnlichen Projekten;

→ Delphi-Methode (Mehrere Experten unabhängig)

▷ Algorithmisches Verfahren: Mathematische Formeln auf Basis messbarer Funktionen

↳ Function-Point-Methode (FPM): Basiert auf Anforderungen

↳ COCOMO: Basiert auf geschätzten LOC

Function-Point-Methode (FPM)

(i) Jede Anforderung einem von 5 Typen zuordnen:

↳ External Input (EI)

↳ External Output (EO)

↳ External Query (EQ)

↳ Internal Logical File (ILF)

↳ External Logical File (ELF)

(ii) Komplexität bestimmen

↳ einfach/mittel/komplex ⇒ anhand Records/Referenzen + Data Elements:

EI	3/4/6
EO	4/5/7
EQ	3/4/6
ILF	7/10/15
ELF	5/7/10

(iii) Summe = Unadjusted Function Points (UFP)

(iv) Einflussfaktoren (S2, 0-60) → S3 = 0,7 + 0,01 · S2 (Bereich 0,7-1,3)

(v) Total FP = UFP · S3

(vi) Aufwand aus Erfahrungskurve/Tabelle ableiten

COCOMO (COnstructive COst MOdel)

Formel: $A = a \cdot (KDSI)^b \cdot m(X)$

↳ KDSI = Kilo Delivered Source Instructions (1000 LOC ohne Kommentare)

↳ m(X) = Anpassungsfaktoren (im Basismodell → 1)

Koeffizienten nach Schwierigkeit:

↳ einfach: <50 KSLOC; a = 2,5; b = 1,05

↳ normal: 50-300 KSLOC; a = 3,0; b = 1,12

↳ schwierig: >300 KSLOC; a = 3,6; b = 1,2

⇒ Ergebnis A: Personenmonate (PM)

Umrechnung Function Points (FP) → Lines Of Code (LOC): Java ≈ 50 LOC/FP

Schwächen: LOC-Schätzung schwierig vor Implementierung, sprachabhängig, historische Daten nötig, Basismodell stark vereinfacht

Werkzeuge & Case

- Joel-Test (12 Punkte):
  → Versionskontrolle? Ein-Schritt-Build? Täglicher Build?
  Fehlerdatenbank? Bugs vor Neuem? Zeitplan? Spezifikation?
  Ruhige Arbeitsumgebung? Beste Tools? Tester?
  Code im Interview? Usability Test?
- CASE-Tools: IDE, Versionskontrolle (Git), CI/CD, Bugtracker, Testtools, Dokumentation
- Docker: Containisierung für reproduzierbare Umgebung, Entkopplung von Infrastruktur

MDSD (Modellgetriebene Softwareentwicklung)

Techniken, die aus formalen Modellen automatisiert lauffähige Software erzeugen

- Modelle: zentrale Artefakte
- Generierung von Code, Tests, Docs

Nicht: Rumpf-Code generieren und händisch weiterarbeiten
Sondern: Templates + Modell → fertiger Code, wiederholbar

Mensch

- Produktivität: Nicht linear mit Teamgröße
  ↳ Interaktionsverlust + Integrationskosten
- Brooks' Law: „Adding manpower to a late software makes it later"
- Druck & Leistung: Moderater Druck → beste Leistung
  Zu viel Druck → Qualitätsverlust
- Anti-Patterns: Mañana (Entscheidungsscheu), Kühlschranktür (Wissen verschwindet, wenn Mitarbeiter geht)